

SmartBIS eGov Backend 구축 기록 v1.0

이 문서는 **SmartBIS_jangsung** 신규 프로젝트에서 PostgreSQL과 Spring Boot 내장 WAS 컨테이너를 구축하고 검증한 1~12단계 기록이다.

0. 이 문서를 사용하는 방법

각 단계는 다음 순서로 읽고 실행한다.

1. 해당 단계의 목적을 확인한다.
2. 명령을 Git Bash에서 실행한다.
3. 정상 결과와 실제 결과를 비교한다.
4. 정상 결과가 확인된 후에만 다음 단계로 진행한다.

이 문서의 명령은 Windows의 Git Bash와 Podman Desktop 환경을 기준으로 한다. 명령 블록 안의 **EOF**는 파일 작성이 끝나는 위치이므로 임의로 수정하지 않는다. Markdown 문서의 코드펜스인 `java````나 text`````를 실제 소스 파일 안에 입력하면 안 된다.

이 기록은 기능 구현 완료 문서가 아니라, 서버 실행 기반이 정상인지 확인하는 초기 구축 기록이다. 아직 Nginx TLS, Node watcher, Android 통신, 실제 콘텐츠 API는 다음 단계의 작업이다.

1. 최종 구조

```
SmartBIS_jangsung/
├──compose.yaml
├──.env
├──.gitignore
├──server/
│   ├──egov-backend/
│   │   ├──Dockerfile
│   │   ├──pom.xml
│   │   └──src/main/
│   │       ├──java/com/smartbis/backend/SmartBisApplication.java
│   │       └──resources/application.properties
│   ├──nginx/
│   │   └──certs/
│   ├──node-watcher/
│   ├──upload-store/
│   └──client-android/
```

2. 구축 원칙

- Spring Boot 내장 WAS 사용
- 외부 Tomcat 설치 및 WAR 배포 사용 안 함
- Spring Boot, PostgreSQL, Node watcher, Nginx는 컨테이너 실행
- Spring Boot 내부 포트: **8080**
- PostgreSQL 내부 포트: **5432**

- 외부 공개 포트: Nginx 8443:443만 허용
- 컨테이너 간 통신은 서비스명 사용
- localhost는 컨테이너 간 DB 주소로 사용하지 않음

왜 외부 Tomcat을 사용하지 않는가

Spring Boot는 JAR 안에 내장 Tomcat을 포함할 수 있다. 따라서 별도의 Tomcat 설치, Eclipse의 서버 런타임 등록, webapps에 WAR 복사, context path /sht_webapp 설정이 필요하지 않다.

실행 흐름은 다음과 같다.

```
smartbis-backend.jar
  ↓
Spring Boot 컨테이너
  ↓
내장 Tomcat 8080
```

로그에 Tomcat started on port 8080이 표시되더라도 이는 외부 Tomcat 설치를 의미하지 않는다. Spring Boot 프로세스 안에서 내장 WAS가 실행되었다는 의미다.

왜 컨테이너 간 주소에 서비스명을 사용하는가

컨테이너 안에서 localhost는 해당 컨테이너 자신을 의미한다. 따라서 Spring Boot에서 다음과 같이 설정하면 PostgreSQL이 아니라 Spring Boot 컨테이너 자신에게 접속하게 된다.

```
spring.datasource.url=jdbc:postgresql://localhost:5432/...
```

Compose가 제공하는 내부 DNS를 사용해 PostgreSQL 서비스명인 postgres-db로 연결해야 한다.

```
spring.datasource.url=jdbc:postgresql://postgres-db:5432/...
```

2-1. 포트와 주소의 의미

구분	주소	용도	외부 공개
PostgreSQL	postgres-db:5432	Spring Boot의 DB 연결	아니오
Spring Boot	egov-backend:8080	Nginx의 내부 API 전달	아니오
Nginx	localhost:8443	PC 테스트용 HTTPS	예
Android Emulator	10.0.2.2:8443	호스트의 8443 접근	예

초기 단계에서는 Nginx가 아직 없으므로 외부에서 Spring Boot 8080으로 접근하지 않는다. 먼저 Compose 내부에서 Health API를 검증한다.

3. 기본 환경 파일

`.env`:

```
POSTGRES_DB=smartbis_db
POSTGRES_USER=smartbis_user
POSTGRES_PASSWORD=change_this_password
TZ=Asia/Seoul
```

`.gitignore`에는 `.env`, 인증서 개인키, `target/`, `node_modules/`를 포함한다.

4. PostgreSQL Compose 구성

초기 `compose.yaml`은 PostgreSQL만 포함하여 인프라를 먼저 검증했다.

```
name: smartbis_jangsung

services:
  postgres-db:
    image: docker.io/library/postgres:15-alpine
    env_file:
      - .env
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      TZ: ${TZ}
    volumes:
      - db-data:/var/lib/postgresql/data
    expose:
      - "5432"
    restart: unless-stopped

volumes:
  db-data:
```

실행:

```
cd /d/PROJECT/SmartBIS_jangsung
podman compose up -d postgres-db
podman compose ps
```

`expose`는 Compose 내부 통신을 문서화하고 허용하는 설정이며, 호스트의 포트를 여는 `ports`와 다르다. 이 프로젝트에서는 DB를 외부에 공개하지 않기 위해 `ports: "5432:5432"`를 사용하지 않는다.

5. PostgreSQL 연결 검증

```
podman compose exec postgres-db pg_isready -U smartbis_user -d smartbis_db
podman compose exec postgres-db psql -U smartbis_user -d smartbis_db -c "SELECT
current_database(), current_user, now();"
```

정상 기준:

```
/var/run/postgresql:5432 - accepting connections
smartbis_db | smartbis_user | 현재 시간
```

이 검증은 단순히 컨테이너가 떠 있지만 확인하는 것이 아니다. PostgreSQL 프로세스가 요청을 받을 준비가 되었는지, 지정한 DB가 생성되었는지, 지정한 사용자가 인증되는지를 확인한다.

6. Spring Boot 프로젝트 파일

필수 파일:

```
server/egov-backend/
├─pom.xml
└─src/main/
    ├─java/com/smartbis/backend/SmartBisApplication.java
    └─resources/application.properties
```

`SmartBisApplication.java`에는 Markdown 코드펜스가 포함되면 안 된다.

```
package com.smartbis.backend;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SmartBisApplication {
    public static void main(String[] args) {
        SpringApplication.run(SmartBisApplication.class, args);
    }
}
```

`@SpringBootApplication`은 컴포넌트 스캔과 자동 설정의 시작점이다. 이 클래스가 없거나 패키지 위치가 어긋나면 JAR가 빌드되더라도 Spring Boot 애플리케이션이 정상 시작되지 않을 수 있다.

7. Spring Boot 설정

```
spring.application.name=smartbis-backend
server.port=8080
```

```
server.servlet.context-path=/api

spring.datasource.url=jdbc:postgresql://postgres-db:5432/${POSTGRES_DB}
spring.datasource.username=${POSTGRES_USER}
spring.datasource.password=${POSTGRES_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver

management.endpoints.web.exposure.include=health,info
management.endpoint.health.show-details=always
```

중요: `${POSTGRES_DB}`와 같은 변수에 역슬래시가 들어가면 안 된다.

`server.servlet.context-path=/api`를 설정했기 때문에 Actuator Health 주소는 컨테이너 내부 기준으로 다음과 같다.

```
http://egov-backend:8080/api/actuator/health
```

나중에 Nginx가 외부 8443을 받으면 Android는 다음 주소를 사용한다.

```
https://10.0.2.2:8443/api/actuator/health
```

8. Spring Boot Dockerfile

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/smartbis-backend.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

`COPY target/smartbis-backend.jar app.jar`는 Docker 이미지 빌드 전에 JAR가 존재해야 한다는 뜻이다. 따라서 Maven 빌드가 먼저 성공해야 한다. 이 파일은 소스 코드를 컴파일하지 않고 이미 만들어진 JAR를 실행 이미지에 넣는 역할만 한다.

9. Maven 빌드

호스트에 Maven이 없어도 Maven 컨테이너로 빌드할 수 있다. Windows Git Bash에서는 `MSYS_NO_PATHCONV=1`을 사용한다.

```
cd /d/PROJECT/SmartBIS_jangsung/server/egov-backend

MSYS_NO_PATHCONV=1 podman run --rm -v "D:/PROJECT/SmartBIS_jangsung/server/egov-backend:/app" -w /app docker.io/library/maven:3.9-eclipse-temurin-17 mvn clean package -DskipTests
```

성공 기준:

```
BUILD SUCCESS
target/smartbis-backend.jar 생성
```

빌드 디렉터리는 호스트 프로젝트와 컨테이너 `/app`을 연결한다. `--rm`은 빌드가 끝난 Maven 임시 컨테이너를 자동 제거하고, 실제 결과물인 `target/`은 호스트에 남긴다.

Windows Git Bash에서 `-w /app`이 `C:/Program Files/Git/app`으로 바뀌는 오류가 발생할 수 있다. 이때 `MSYS_NO_PATHCONV=1`은 Git Bash의 자동 경로 변환을 끄는 설정이다.

10. Backend Compose 서비스 추가

최종 `compose.yaml`에는 다음 서비스를 포함한다.

```
egov-backend:
  build:
    context: ./server/egov-backend
  env_file:
    - .env
  expose:
    - "8080"
  depends_on:
    - postgres-db
  restart: unless-stopped
```

검증:

```
cd /d/PROJECT/SmartBIS_jangsung
podman compose config
podman compose build egov-backend
```

`build.context`는 Dockerfile과 `target/smartbis-backend.jar`를 찾는 기준 디렉터리다. 따라서 프로젝트 루트에서 실행하더라도 `./server/egov-backend`를 기준으로 이미지를 만든다.

11. Backend 컨테이너 실행

```
podman compose up -d postgres-db egov-backend
podman compose ps
podman compose logs --tail=100 egov-backend
```

정상 로그:

```
Tomcat started on port 8080 (http) with context path '/api'
Started SmartBisApplication
```

여기서 Tomcat은 별도 설치된 외부 Tomcat이 아니라 Spring Boot 내장 WAS다.

`depends_on`은 PostgreSQL 컨테이너가 먼저 시작되도록 순서를 지정한다. 다만 컨테이너 시작과 DB가 실제로 연결을 받을 준비가 된 시점은 다를 수 있다. Spring Boot가 재시작 정책을 가지고 있거나 애플리케이션에 재시도 로직이 있어야 운영 환경에서 더 안정적이다. 이번 검증에서는 PostgreSQL을 먼저 실행한 뒤 Spring Boot를 실행했으므로 정상 기동했다.

12. 컨테이너 내부 API 검증

호스트에 8080을 공개하지 않았으므로 Compose 네트워크의 임시 curl 컨테이너를 사용한다.

```
podman run --rm --network smartbis_jangsung_default
docker.io/curlimages/curl:8.10.1 http://egov-backend:8080/api/actuator/health
```

정상 응답:

```
{"status": "UP", "components": {"db": {"status": "UP"}}
```

이 응답으로 다음을 동시에 검증했다.

- Spring Boot 내장 WAS 실행
- Compose 내부 DNS에서 `egov-backend` 확인
- `/api` context path 정상
- PostgreSQL 연결 정상
- Actuator health 정상

`db.status=UP`이 특히 중요하다. `status=UP`만 있고 DB 구성요소가 없거나 `DOWN`이면 Spring Boot는 실행 중이어도 데이터베이스 연결은 실패한 상태일 수 있다.

15. 재구축 시 빠른 실행 순서

이미 파일이 준비되어 있는 경우 다음 순서로 재구축한다.

```
cd /d/PROJECT/SmartBIS_jangsung
podman compose config
podman compose build egov-backend
podman compose up -d postgres-db egov-backend
podman compose ps
podman compose logs --tail=100 egov-backend
podman run --rm --network smartbis_jangsung_default
docker.io/curlimages/curl:8.10.1 http://egov-backend:8080/api/actuator/health
```

정상 판정은 다음 네 가지를 모두 만족해야 한다.

```
postgres-db: Up
egov-backend: Up
Started SmartBisApplication
health 응답의 status 및 db.status가 UP
```

16. 실패 시 점검 순서

컨테이너가 바로 종료됨

```
podman compose ps -a
podman compose logs --tail=200 egov-backend
```

target/smartbis-backend.jar를 찾지 못함

```
ls -lh server/egov-backend/target/smartbis-backend.jar
```

없으면 Maven 컨테이너 빌드를 다시 실행한다.

DB 연결 실패

```
podman compose exec postgres-db pg_isready -U smartbis_user -d smartbis_db
podman compose logs --tail=100 postgres-db
```

application.properties에서 DB 주소가 localhost가 아닌 postgres-db인지 확인한다.

Compose 설정이 예상과 다름

```
podman compose config
```

특히 `egov-backend`에 `expose: 8080`이 있는지, 외부 `ports: 8080:8080`이 불필요하게 추가되지 않았는지 확인한다.

13. 다음 구축 단계

다음 순서로 진행한다.

1. 사설 CA 및 서버 인증서 생성
2. Nginx `8443:443` 설정
3. Nginx → `egov-backend:8080` HTTPS 경로 검증
4. Node watcher 컨테이너 추가
5. Android Emulator의 `https://10.0.2.2:8443` 통신 검증

14. 주요 오류와 해결

Maven 명령 없음

```
bash: mvn: command not found
```

호스트 Maven 대신 Maven 컨테이너 빌드 명령을 사용한다.

Git Bash 경로 변환 오류

```
workdir "C:/Program Files/Git/app" does not exist
```

`MSYS_NO_PATHCONV=1`을 명령 앞에 붙이고 Windows 절대 경로를 사용한다.

Java 파일에 코드펜스 포함

````java와 ````는 파일에 입력하지 않는다. 실제 Java 파일에는 Java 소스만 있어야 한다.

컨테이너 간 localhost 사용

잘못된 설정:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/...
```

올바른 설정:

```
spring.datasource.url=jdbc:postgresql://postgres-db:5432/...
```